

Lab 10 — Genetic Algorithm to Maximise a Function

Aim

To implement a simple Genetic Algorithm (GA) to maximise the function $f(x) = x^2$ over integers from 0 to 31 using 5-bit chromosome encoding.

Algorithm / Procedure

1. Represent each candidate solution x as a 5-bit binary chromosome.
2. Generate an initial random population.
3. Decode each chromosome into its integer value.
4. Calculate the fitness using:

$$f(x) = x^2$$

5. Select parents using roulette-wheel selection.
6. Perform single-point crossover to generate offspring.
7. Apply mutation to randomly change bits.
8. Preserve the best chromosome using elitism.
9. Repeat the process for the specified number of generations.
10. Display the best chromosome, its value of x , and its fitness.

Program

```
import random

# Decode binary chromosome
def decode(chrom):
    return int(chrom, 2)

# Fitness function
def fitness(chrom):
    x = decode(chrom)
    return x ** 2

# Genetic Algorithm
```

```
def ga(pop_size=6, generations=20, mut_rate=0.05):

    random.seed(1)

    # Initial population
    population = [

        ".join(random.choice('01') for _ in range(5))

        for _ in range(pop_size)

    ]

    for gen in range(generations):

        # Calculate fitness

        fitnesses = [fitness(c) for c in population]

        # Preserve best chromosome (elitism)

        best_chromosome = max(population, key=fitness)

        total = sum(fitnesses)

        # Avoid division by zero

        if total == 0:

            probs = [1 / len(population)] * len(population)

        else:

            probs = [f / total for f in fitnesses]

        new_population = [best_chromosome]

        # Selection and crossover

        while len(new_population) < pop_size:

            p1, p2 = random.choices(

                population,

                weights=probs,

                k=2

            )
```

```

# Single-point crossover

point = random.randint(1, 4)

c1 = p1[:point] + p2[point:]
c2 = p2[:point] + p1[point:]

new_population.append(c1)

if len(new_population) < pop_size:

    new_population.append(c2)

# Mutation

population = []

for chromosome in new_population:

    mutated = ".join(

        bit if random.random() > mut_rate

        else str(1 - int(bit))

        for bit in chromosome

    )

    population.append(mutated)

# Final best solution

best = max(population, key=fitness)

return decode(best), fitness(best)

# Run Genetic Algorithm

x, fx = ga()

print("Best x =", x)

print("f(x) =", fx)

```

OUTPUT



```

❏ Best x = 31

f(x) = 961

```

Result

The Genetic Algorithm was successfully implemented using 5-bit binary encoding, roulette-wheel selection, single-point crossover, mutation, and elitism. The algorithm converged toward the maximum value of $f(x) = x^2$, which occurs at $x = 31$, giving $f(31) = 961$.